

Diseño y buenas prácticas para el desarrollo de aplicaciones reactivas basadas en el Manifiesto Reactivo

E.J Laferre Guevara
7690222644 Universidad Mariano Gálvez
Seminario de Tecnologías de la Información
elaferre1@miumg.edu.gt

Resumen

Este ensayo explora la arquitectura reactiva como enfoque de diseño de sistemas cuyo objetivo central es mantener la responsividad bajo cualquier condición, ya sea alta carga, fallos parciales o cambios en la demanda, en respuesta a las exigencias del usuario moderno, que depende del software y no tolera demoras ni interrupciones.

El Manifiesto Reactivo formaliza este enfoque mediante cuatro pilares: sistemas responsivos, resilientes, elásticos y orientados a mensajes, donde la comunicación asíncrona sostiene la elasticidad y la resiliencia del sistema. A partir de este marco, se analizan los patrones de diseño propuestos en los Reactive Principles, entre ellos la partición del estado, la comunicación mediante hechos inmutables, el aislamiento de mutaciones, la coordinación del flujo de datos, la localización del estado y la observación de las comunicaciones, cada uno vinculado a un pilar del manifiesto.

Finalmente, se presentan buenas prácticas para el desarrollo de aplicaciones reactivas: adopción gradual del enfoque, uso de frameworks especializados, gestión de la contrapresión, mecanismos de tolerancia a fallos, pruebas exhaustivas, monitoreo del rendimiento y actualización constante frente a la evolución de las herramientas del paradigma.

Palabras clave: Arquitectura reactiva, Manifiesto Reactivo, sistemas resilientes, programación asíncrona, tolerancia a fallos, contrapresión (backpressure), elasticidad, procesamiento de flujos de datos

Fundamentos de la arquitectura reactiva

Según (Güzel, 2021), la arquitectura reactiva surge como respuesta a un cambio profundo en la forma en que se construyen y usan los sistemas de software actuales. Mientras que hace una década las aplicaciones se instalaban en pocos nodos y manejaban volúmenes de datos relativamente estáticos, hoy se despliegan en cientos o miles de máquinas, procesan datos que cambian constantemente, y ya no es aceptable la indisponibilidad del servicio. Este cambio técnico es, en el fondo, consecuencia de algo más profundo: la evolución en las expectativas de los usuarios, quienes dependen del software para su vida cotidiana y no toleran demoras ni interrupciones, bajo riesgo de migrar hacia alternativas de la competencia. Por lo tanto, la arquitectura reactiva puede definirse como un enfoque de diseño de sistemas cuyo objetivo central es ofrecer una experiencia responsiva bajo cualquier condición, ya sea alta carga, fallos parciales o cambios en la demanda.

Para formalizar este objetivo, en 2014 se publicó el Manifiesto de Sistemas Reactivos, que puede entenderse como una extensión concreta de la arquitectura reactiva: mientras esta última plantea el objetivo general de la responsividad, el manifiesto traduce ese objetivo en cuatro características de diseño necesarias para alcanzarlo. Un sistema reactivo debe ser responsivo, proporcionando

respuestas oportunas y consistentes, incluso al informar errores o situaciones inesperadas; resiliente, manteniendo su capacidad de respuesta ante fallos mediante el aislamiento y la contención de errores en componentes específicos; elástico, ajustando dinámicamente sus recursos ante variaciones en la carga de trabajo, evitando cuellos de botella; y orientado a mensajes, comunicando sus componentes mediante intercambio asíncrono, lo cual reduce el acoplamiento, favorece el aislamiento y permite un uso más eficiente de los recursos (Bonér et al., 2014).

Estos cuatro principios están interrelacionados: la comunicación orientada a mensajes favorece el desacoplamiento entre componentes y facilita tanto la resiliencia como la elasticidad, lo cual permite finalmente sostener la responsividad del sistema. Por esta razón, el manifiesto no debe entenderse como una simple lista de características aisladas, sino como un marco que operacionaliza el objetivo central de la arquitectura reactiva: mantener sistemas confiables, adaptables y capaces de responder de forma oportuna ante las exigencias actuales.

Componentes y patrones de la arquitectura reactiva

Además de los cuatro pilares del Manifiesto Reactivo, existen patrones de diseño que orientan la construcción práctica de sistemas reactivos: soluciones reutilizables a problemas recurrentes del diseño distribuido, que traducen los principios del manifiesto en decisiones concretas de implementación. Estos patrones fueron formalizados por Bonér et al. (2022) en *The Reactive Principles*, complemento técnico al Manifiesto Reactivo.

Entre los principales se encuentran: *particionar el estado*, que divide el estado de la aplicación en fragmentos independientes para distribuir la carga entre nodos; *comunicar hechos*, que privilegia el uso de eventos inmutables sobre el estado mutable como medio de comunicación entre componentes; *aislar mutaciones*, que contiene el estado mutable en compartimentos separados (*bulkheads*) para evitar que un fallo se propague en cascada; *coordinar el flujo de datos*, que permite a cada componente controlar su propio ritmo de consumo mediante mecanismos de *back-pressure*; *localizar el estado*, que coubica datos y procesamiento para reducir la latencia; y *observar las comunicaciones*, que monitorea el intercambio de mensajes entre componentes para detectar cuellos de botella y sostener decisiones de elasticidad (Bonér et al., 2022).

En conjunto, estos patrones materializan los pilares del manifiesto: la partición y localización del estado favorecen la elasticidad, el aislamiento de mutaciones refuerza la resiliencia, y la comunicación por hechos junto con la coordinación del flujo de datos sostienen la orientación a mensajes.

Buenas prácticas para el desarrollo de aplicaciones reactivas

El desarrollo de aplicaciones reactivas requiere prácticas que permitan gestionar la asincronía, controlar el flujo de datos, responder ante fallos y mantener un rendimiento adecuado. La aplicación de estas recomendaciones contribuye a crear sistemas más eficientes, resilientes y adaptables.

Adoptar el enfoque reactivo de manera gradual

Se recomienda comenzar con componentes pequeños y manejables antes de aplicar la programación reactiva a sistemas complejos. Esto permite comprender los flujos asíncronos, identificar dificultades y reducir los riesgos durante la implementación. Posteriormente, el enfoque reactivo

puede incorporarse progresivamente en otras áreas de la aplicación (AiimTech Software System Pvt. Ltd., 2024; Sousa, 2025).

Utilizar herramientas y *frameworks* adecuados

El uso de herramientas consolidadas facilita el desarrollo y evita crear soluciones desde cero. Tecnologías como Project Reactor, RxJava, Spring WebFlux, Akka y Vert.x proporcionan mecanismos para trabajar con eventos, operaciones asíncronas y flujos de datos. La selección debe realizarse de acuerdo con las necesidades y características de cada proyecto (Saeed, 2025; Sousa, 2025).

Gestionar el flujo de datos y la contrapresión

La contrapresión o *backpressure* ocurre cuando un productor genera datos a una velocidad mayor que la capacidad de procesamiento del consumidor. Para evitar la sobrecarga, pueden utilizarse mecanismos como el almacenamiento temporal de datos, la limitación de la velocidad de procesamiento y el control de la carga. Estas estrategias ayudan a mantener la estabilidad y la capacidad de respuesta del sistema (Saeed, 2025; Sousa, 2025).

Diseñar sistemas resilientes ante fallos

Los sistemas reactivos deben considerar que los errores pueden ocurrir. Por ello, es importante implementar reintentos controlados, respuestas alternativas o *fallbacks*, e interruptores de circuito o *circuit breakers*. Estas estrategias ayudan a evitar que un fallo se propague y afecte el funcionamiento completo de la aplicación (Saeed, 2025).

Realizar pruebas exhaustivas

Las pruebas permiten verificar el comportamiento de los flujos reactivos ante errores, casos límite y diferentes niveles de carga. Debido a la naturaleza asíncrona de estos sistemas, es importante realizar pruebas unitarias, de integración y de estrés para comprobar su funcionamiento en distintas condiciones (Saeed, 2025; AiimTech Software System Pvt. Ltd., 2024).

Monitorear y optimizar el rendimiento

El monitoreo continuo permite detectar errores, identificar cuellos de botella y evaluar el rendimiento de la aplicación. Algunas métricas importantes son los tiempos de respuesta, la cantidad de solicitudes procesadas, las tasas de error y el consumo de recursos. La información obtenida facilita la optimización de los flujos y ayuda a mantener la capacidad de respuesta del sistema (Saeed, 2025; AiimTech Software System Pvt. Ltd., 2024).

Mantener un aprendizaje y actualización continuos

Las herramientas y prácticas relacionadas con la programación reactiva evolucionan constantemente. Por esta razón, es recomendable consultar documentación, participar en comunidades técnicas y revisar recursos especializados. La actualización continua permite conocer nuevas soluciones y tomar decisiones más adecuadas durante el desarrollo (Sousa, 2025).

En conjunto, estas prácticas permiten aplicar los principios del Manifiesto Reactivo y favorecen la creación de aplicaciones más responsivas, resilientes, elásticas y capaces de adaptarse a diferentes condiciones de funcionamiento.

Observaciones y comentarios

Conclusiones

1. ...

Referencias

- AiimTech Software System Pvt. Ltd. (2024, junio). *Understanding Reactive Programming: Principles and Applications in Modern Development*. Medium. Consultado el 30 de julio de 2026, desde https://medium.com/@business_23442/understanding-reactive-programming-principles-and-applications-in-modern-development-8496f903ef68
- Bonér, J., Farley, D., Kuhn, R., & Thompson, M. (2014, septiembre). *The Reactive Manifesto* [Autores en orden alfabético con contribuciones equivalentes] (2.0). Consultado el 30 de julio de 2026, desde <https://www.reactivemanifesto.org/es>
- Bonér, J., Kuhn, R., Christensen, B., Vlugter, P., Long, J., Bykov, S., Hindman, B., Vernon, V., Escoffier, C., Roper, J., Behrendt, M., Thorup, K. K., Breck, C., Buijze, A., Collison, D., Klang, V., Hale, B., Gury, S., Jewell, T., . . . Degnan, R. (2022, junio). *The Reactive Patterns* (1.0). The Reactive Principles. Consultado el 30 de julio de 2026, desde <https://www.reactiveprinciples.org/patterns/index.html>
- Güzel, C. (2021, mayo). *Reactive Architecture*. Medium. Consultado el 30 de julio de 2026, desde <https://medium.com/big-data-cloud-computing-and-distributed-systems/reactive-architecture-i-5652f944f8fb>
- Saeed, L. (2025, junio). *Reactive Programming in Java*. Medium. Consultado el 30 de julio de 2026, desde <https://www.azul.com/blog/reactive-programming-in-java/>
- Sousa, A. (2025, agosto). *Java Reactive Programming: An In-Depth Analysis*. Consultado el 30 de julio de 2026, desde <https://www.rethink.de/en/blog/java-reactive-programming-an-in-depth-analysis>

URL del repositorio: [Repositorio del código Latex](#)